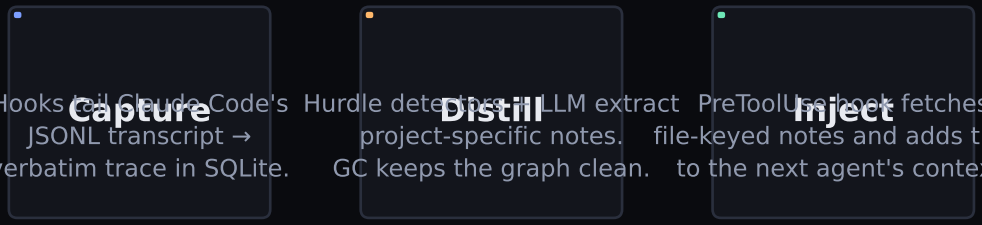
NM · Context Cloud

Shared memory for coding agents

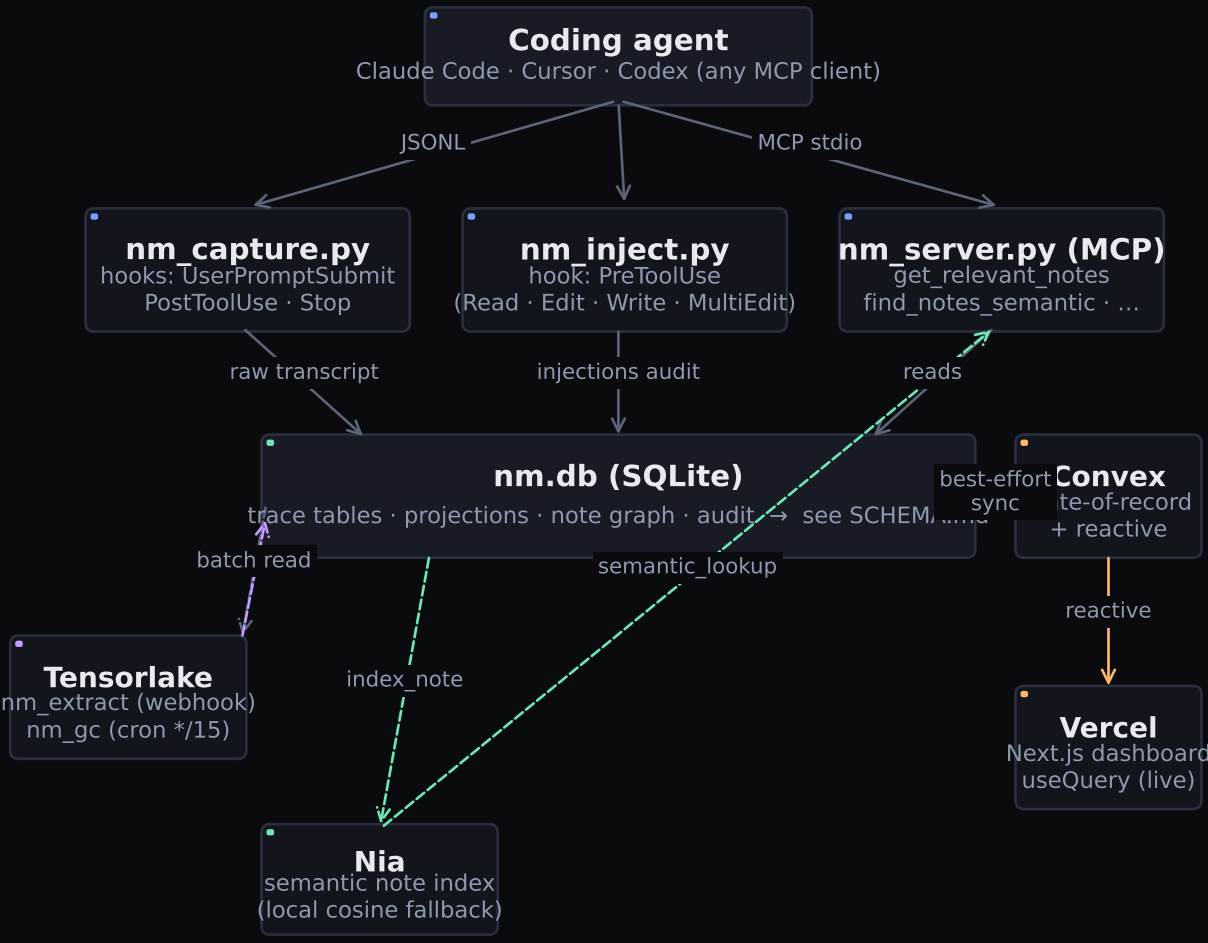
An MCP-served note graph that learns from every coding session in your org and injects the relevant lesson into the next agent that touches the same file.



Always-On Agents track · Nia · Tensorlake · Convex · Vercel

End-to-end data flow

from a Claude Code keystroke to the live dashboard



solid = inline (latency-critical) · dashed = best-effort (network, fail-open)

Three always-on agents

three trigger types — deliberate hit on the Background Execution rubric

Note Manager

fires on Stop / SubagentStop

`WEBHOOK`

What it does

Reads session's transcript
Runs 7 hurdle-detection signals
Expands signal clusters → windows
LLM distills each window into a 4-field note (gpt-4o-mini)
Persists notes + edges + hurdles

Where it runs
tensorlake/note_manager.py
(local CLI: nm_extract.py)

Guardian

fires on every PreToolUse

`EVENT (per-injection)`

What it does

Scores candidate notes for the current session's recent context
Resolves contradictions
Enforces per-injection token budget
Logs accept / reject reasons to the injections audit table

Where it runs
tensorlake/guardian.py
(owned by another teammate)

GC

scheduled long-term hygiene

`CRON */15 * * * *`

What it does

Decay: importance halves every 7 days of idleness
Merge: Jaccard 0.6 / cosine 0.5 on overlapping files
Prune: importance < 0.10 → invalidate (soft delete)

Where it runs
tensorlake/gc.py
(local CLI: nm_gc.py)

(planned · slot reserved)

All three share state through the same SQLite tables and Convex mirror.

Removing background execution or memory genuinely breaks the demo — the rubric bar.

Database schema — four modules

OpenInference / OTel-shaped trace · projections · product graph · audit

TRACE

verbatim chat capture

```
sessions       — one row per Claude Code session
messages       — one row per transcript entry (≈ span)
content_blocks — text · thinking · tool_use · tool_result · image
ingest_state   — per-transcript line offset (incremental)
```

INDEX (projections)

fast queries derived from content_blocks

```
tool_calls   — tool_use joined to its tool_result
file_touches — (tool_call, canonical path) for O(idx) lookup
```

NOTE (product graph)

bipartite files ↔ notes — what other agents read

```
notes           — id · symptom · root_cause · correction · importance
files           — canonical path registry (lowercase drive, fwd slashes)
file_note_edges — bipartite edge with per-file weight
```

LIFECYCLE (audit)

every state change recorded — drives on-stage metrics

```
hurdles + hurdle_signals — detection windows + per-signal weights
injections               — every PreToolUse match (accepted / filtered)
note_feedback            — useful=true / false from agent or user
gc_actions               — every prune / merge / decay (cron tick proof)
```

Sponsor stack mapping

what each platform owns — and what local fallback keeps the demo alive

Nia

semantic note index

Owns: every persisted note is registered with a nia… — find_notes_semantic MCP tool searches at lookup time

Fallback: *local cosine ranker over notes table (no API key needed)*

Always-On track sponsor

Tensorlake

background sandboxed agent runtime

Owns: Note Manager (webhook) + GC (cron */15 * * * *) — three trigger types = three different rubric proofs

Fallback: *python nm_extract.py · python nm_gc.py --loop*

Always-On track co-sponsor

Convex

state-of-record + live reactive backend

Owns: product graph (notes / edges / files / inbox / GC proposals) — WebSocket reactivity drives the live dashboard

Fallback: *SQLite stays authoritative; sync is best-effort*

Statefulness 25% · live demo proof

Vercel

deployed dashboard URL (submission requirement)

Owns: Next.js 15 + ConvexProvider; useQuery hooks for live data — dashboard/ subdir, deploy via npx vercel

Fallback: *nm_dashboard.py on localhost:8765 (not valid for submission)*

Demo & Presentation 10% · public URL required